Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 6

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

На базе любой из ранее выполненных лабораторных работ разработать приложение (C++, C#) с применением паттерна MVC для разделения сложной модели (основной класс) и её представления. Разработать UML-диаграмму классов и диаграмму последовательности. Отчет по лабораторной работе – файл формата pdf. Формат имени файла отчета: _.pdf. В отчет включить построенные диаграммы и исходный код программы (при необходимости и заголовочные файлы). Формат отчета см. в Приложении. Отчет загрузить в LMS.

Работа - ЛР5 «Банкомат»

Диаграмма классов

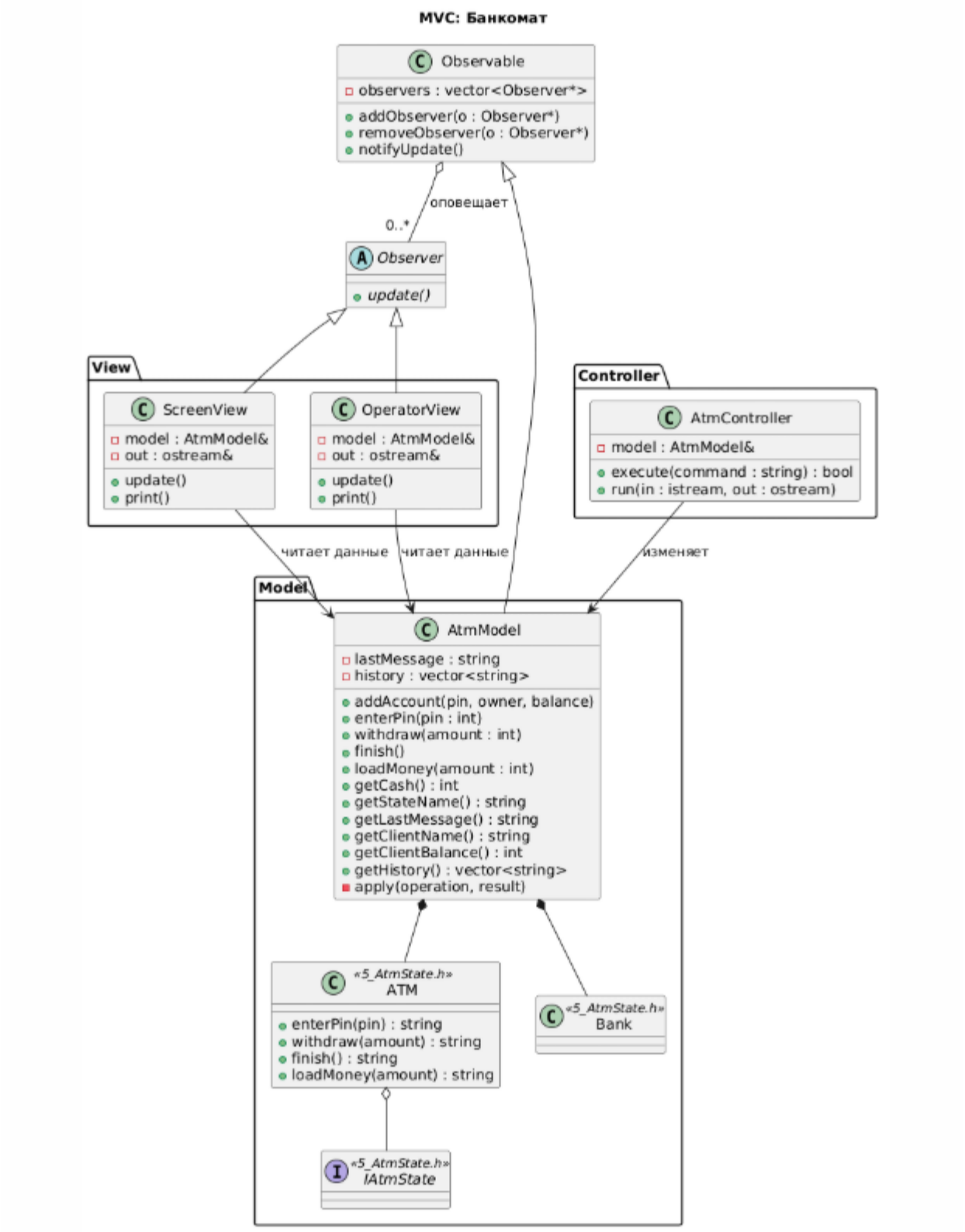
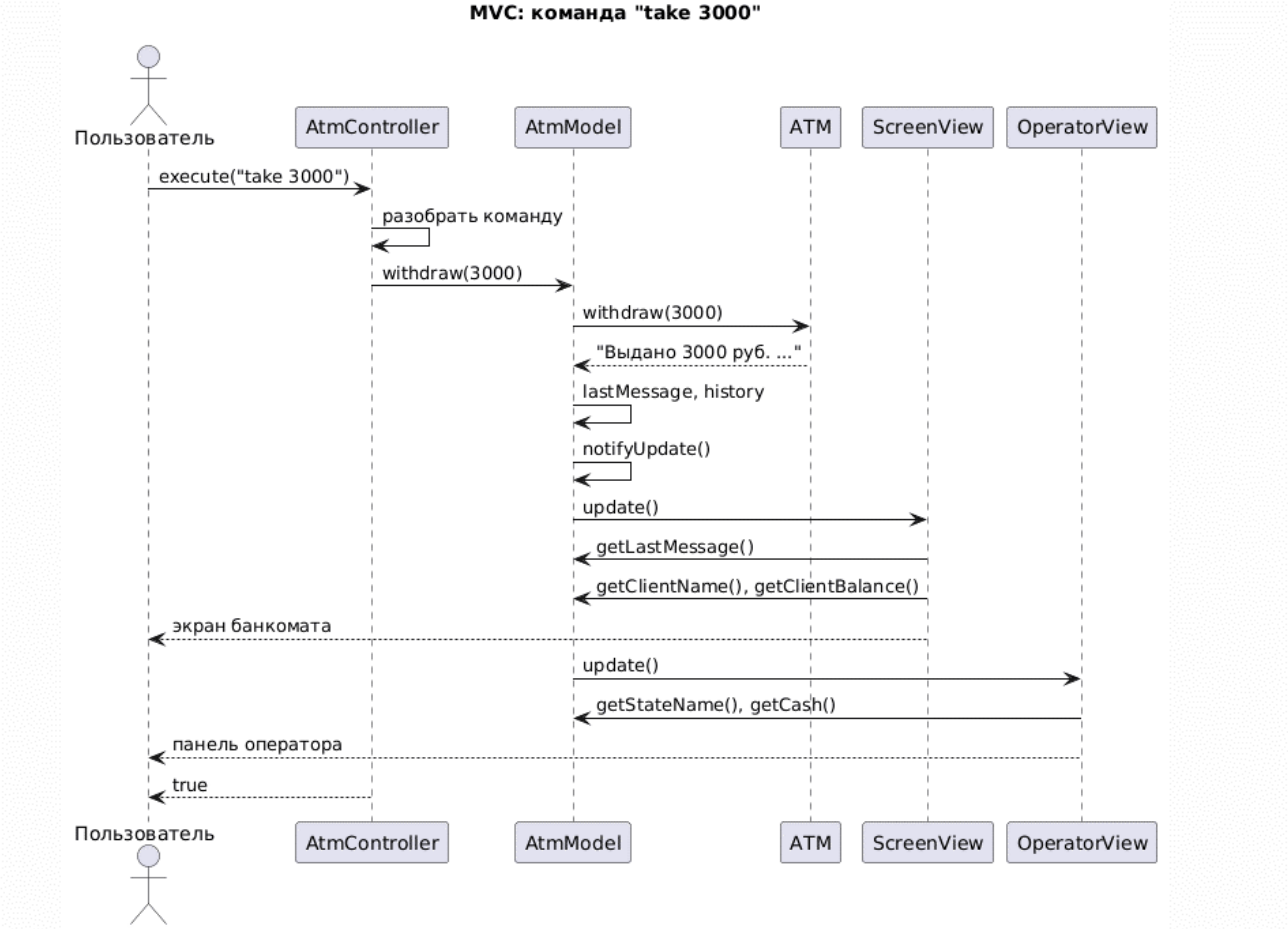


Диаграмма последовательности



Исходный код программы

5_AtmState.h

```
#pragma once

#include <string>
#include <map>
#include <random>

namespace AtmSystem {

// ======================== Банк ========================

// Счет клиента банка
struct Account {
    std::string owner;
    int balance;
};
```

```
// Банк хранит счета; ключ - PIN-код карты (упрощение для лабораторной)
class Bank {
public:
   void addAccount(int pin, const std::string& owner, int balance) {
      accounts[pin] = Account{ owner, balance };
   }

   Account* findAccount(int pin) {
      auto it = accounts.find(pin);
      return it == accounts.end() ? nullptr : &it->second;
   }

private:
   std::map<int, Account> accounts;
};

class ATM;

// ======================== Интерфейс Состояние ========================

// Каждая операция банкомата возвращает сообщение для пользователя
class IAtmState {
public:
   virtual ~IAtmState() {}
   virtual std::string getName() const = 0;
   virtual std::string enterPin(ATM& atm, int pin) = 0;
   virtual std::string withdraw(ATM& atm, int amount) = 0;
   virtual std::string finish(ATM& atm) = 0;
   virtual std::string loadMoney(ATM& atm, int amount) = 0;
};

// ======================== Конкретные состояния ========================

class IdleState : public IAtmState {
public:
   std::string getName() const override { return "Ожидание"; }
   std::string enterPin(ATM& atm, int pin) override;
   std::string withdraw(ATM& atm, int amount) override;
   std::string finish(ATM& atm) override;
   std::string loadMoney(ATM& atm, int amount) override;
};

class AuthenticationState : public IAtmState {
public:
   std::string getName() const override { return "Аутентификация пользователя"; }
   std::string enterPin(ATM& atm, int pin) override;
   std::string withdraw(ATM& atm, int amount) override;
```

```
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

class OperationState : public IAtmState {
public:
    std::string getName() const override { return "Выполнение операций"; }
    std::string enterPin(ATM& atm, int pin) override;
    std::string withdraw(ATM& atm, int amount) override;
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

class BlockedState : public IAtmState {
public:
    std::string getName() const override { return "Заблокирован"; }
    std::string enterPin(ATM& atm, int pin) override;
    std::string withdraw(ATM& atm, int amount) override;
    std::string finish(ATM& atm) override;
    std::string loadMoney(ATM& atm, int amount) override;
};

// ======================== Конечный автомат ========================

// Банкомат хранит текущее состояние и передает ему все операции.
// Объекты состояний создаются один раз и переключаются указателем.
class ATM {
public:
    static const int MAX_ATTEMPTS = 3;

    ATM(const std::string& id, int cash, double noConnectionChance, Bank& bank, unsigned int seed)
        : id(id), cash(cash), noConnectionChance(noConnectionChance), bank(bank),
          random(seed), account(nullptr), attempts(0) {
        state = cash > 0 ? (IAtmState*)&idle : (IAtmState*)&blocked;
    }

    // Операции, которые вызывает пользователь
    std::string enterPin(int pin) { return state->enterPin(*this, pin); }
    std::string withdraw(int amount) { return state->withdraw(*this, amount); }
    std::string finish() { return state->finish(*this); }
    std::string loadMoney(int amount) { return state->loadMoney(*this, amount); }

    std::string getId() const { return id; }
    int getCash() const { return cash; }
    std::string getStateName() const { return state->getName(); }

    // Методы ниже используют классы состояний
```

```
    void setState(IAtmState* newState) { state = newState; }
    IAtmState* getIdleState() { return &idle; }
    IAtmState* getAuthenticationState() { return &authentication; }
    IAtmState* getOperationState() { return &operation; }
    IAtmState* getBlockedState() { return &blocked; }

    void addCash(int amount) { cash += amount; }
    void takeCash(int amount) { cash -= amount; }

    Bank& getBank() { return bank; }
    Account* getAccount() { return account; }
    void setAccount(Account* value) { account = value; }

    int getAttempts() const { return attempts; }
    void setAttempts(int value) { attempts = value; }

    // Есть ли связь с банком (случайное событие)
    bool isBankAvailable() {
        std::uniform_real_distribution<double> dist(0.0, 1.0);
        return dist(random) >= noConnectionChance;
    }

    // Завершение сеанса: сброс данных клиента и переход в нужное состояние
    void endSession() {
        account = nullptr;
        attempts = 0;
        state = cash > 0 ? (IAtmState*)&idle : (IAtmState*)&blocked;
    }

private:
    std::string id;
    int cash;
    double noConnectionChance;
    Bank& bank;
    std::mt19937 random;
    Account* account;
    int attempts;

    IdleState idle;
    AuthenticationState authentication;
    OperationState operation;
    BlockedState blocked;
    IAtmState* state;
};

// ====================== Реализация состояний ======================
```

```
// --- Ожидание ---

inline std::string IdleState::enterPin(ATM& atm, int pin) {
    atm.setState(atm.getAuthenticationState());
    return atm.enterPin(pin);
}

inline std::string IdleState::withdraw(ATM&, int) {
    return "Сначала введите PIN-код";
}

inline std::string IdleState::finish(ATM&) {
    return "Банкомат уже в режиме ожидания";
}

inline std::string IdleState::loadMoney(ATM& atm, int amount) {
    if (amount <= 0) return "Сумма загрузки должна быть больше нуля";
    atm.addCash(amount);
    return "Загружено " + std::to_string(amount) + " руб.";
}

// --- Аутентификация пользователя ---

inline std::string AuthenticationState::enterPin(ATM& atm, int pin) {
    if (!atm.isBankAvailable()) {
        atm.endSession();
        return "Нет связи с банком, попробуйте позже";
    }
    Account* account = atm.getBank().findAccount(pin);
    if (account != nullptr) {
        atm.setAccount(account);
        atm.setState(atm.getOperationState());
        return "PIN-код верный. Здравствуйте, " + account->owner;
    }
    atm.setAttempts(atm.getAttempts() + 1);
    if (atm.getAttempts() >= ATM::MAX_ATTEMPTS) {
        atm.endSession();
        return "Неверный PIN-код. Попытки исчерпаны, карта изъята";
    }
    return "Неверный PIN-код. Осталось попыток: " +
           std::to_string(ATM::MAX_ATTEMPTS - atm.getAttempts());
}

inline std::string AuthenticationState::withdraw(ATM&, int) {
    return "Сначала введите верный PIN-код";
}
```

```
inline std::string AuthenticationState::finish(ATM& atm) {
   atm.endSession();
   return "Операция отменена, заберите карту";
}

inline std::string AuthenticationState::loadMoney(ATM&, int) {
   return "Банкомат занят клиентом, загрузка невозможна";
}

// --- Выполнение операций ---

inline std::string OperationState::enterPin(ATM&, int) {
   return "PIN-код уже введен";
}

inline std::string OperationState::withdraw(ATM& atm, int amount) {
   if (amount <= 0 || amount % 100 != 0) return "Сумма должна быть кратна 100 руб.";
   if (amount > atm.getCash()) return "В банкомате недостаточно денег";
   if (amount > atm.getAccount()->balance) return "Недостаточно средств на счете";
   if (!atm.isBankAvailable()) return "Нет связи с банком, операция не выполнена";

   atm.getAccount()->balance -= amount;
   atm.takeCash(amount);
   std::string message = "Выдано " + std::to_string(amount) + " руб. Остаток на счете: " +
                         std::to_string(atm.getAccount()->balance) + " руб.";
   if (atm.getCash() == 0) {
      atm.endSession();
      message += " Деньги в банкомате закончились, банкомат заблокирован";
   }
   return message;
}

inline std::string OperationState::finish(ATM& atm) {
   atm.endSession();
   return "Работа завершена, заберите карту";
}

inline std::string OperationState::loadMoney(ATM&, int) {
   return "Банкомат занят клиентом, загрузка невозможна";
}

// --- Заблокирован (денег нет) ---

inline std::string BlockedState::enterPin(ATM&, int) {
   return "Банкомат заблокирован: нет денег";
}
```

```
inline std::string BlockedState::withdraw(ATM&, int) {
   return "Банкомат заблокирован: нет денег";
}

inline std::string BlockedState::finish(ATM&) {
   return "Банкомат заблокирован: нет денег";
}

inline std::string BlockedState::loadMoney(ATM& atm, int amount) {
   if (amount <= 0) return "Сумма загрузки должна быть больше нуля";
   atm.addCash(amount);
   atm.setState(atm.getIdleState());
   return "Загружено " + std::to_string(amount) + " руб. Банкомат разблокирован";
}
}
```

6_Practice.cpp

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <algorithm>
#include <random>
#include <clocale>
#include "5_AtmState.h"

using namespace std;
using namespace AtmSystem;

// ======================== Наблюдатель ========================

class Observer {
public:
   virtual ~Observer() {}
   virtual void update() = 0;
};

class Observable {
public:
   void addObserver(Observer* observer) {
      observers.push_back(observer);
   }

   void removeObserver(Observer* observer) {
      observers.erase(remove(observers.begin(), observers.end(), observer), observers.end());
   }
```

```
    void notifyUpdate() {
        for (Observer* observer : observers) {
            observer->update();
        }
    }

private:
    vector<Observer*> observers;
};

// ======================== Модель ========================

// Вся логика работы с данными: банк, банкомат и журнал операций.
// После каждого изменения модель оповещает представления.
class AtmModel : public Observable {
public:
    AtmModel(const string& id, int cash, double noConnectionChance, unsigned int seed)
        : atm(id, cash, noConnectionChance, bank, seed) {}

    void addAccount(int pin, const string& owner, int balance) {
        bank.addAccount(pin, owner, balance);
    }

    void enterPin(int pin) { apply("Ввод PIN", atm.enterPin(pin)); }
    void withdraw(int amount) { apply("Снятие " + to_string(amount), atm.withdraw(amount)); }
    void finish() { apply("Завершение", atm.finish()); }
    void loadMoney(int amount) { apply("Загрузка " + to_string(amount), atm.loadMoney(amount)); }

    string getId() const { return atm.getId(); }
    int getCash() const { return atm.getCash(); }
    string getStateName() const { return atm.getStateName(); }
    string getLastMessage() const { return lastMessage; }
    const vector<string>& getHistory() const { return history; }

    // Имя и баланс клиента, который сейчас обслуживается ("" - клиента нет)
    string getClientName() {
        Account* account = atm.getAccount();
        return account == nullptr ? "" : account->owner;
    }

    int getClientBalance() {
        Account* account = atm.getAccount();
        return account == nullptr ? 0 : account->balance;
    }

private:
    Bank bank;
```

```
    ATM atm;
    string lastMessage;
    vector<string> history;

    void apply(const string& operation, const string& result) {
        lastMessage = result;
        history.push_back(operation + ": " + result);
        notifyUpdate();
    }
};

// ======================== Представления ========================

// Экран банкомата, который видит клиент
class ScreenView : public Observer {
public:
    ScreenView(AtmModel& model, ostream& out) : model(model), out(out) {
        model.addObserver(this);
    }

    ~ScreenView() { model.removeObserver(this); }

    void update() override { print(); }

    void print() {
        out << "+---------- ЭКРАН " << model.getId() << " ----------+" << endl;
        out << "| " << model.getLastMessage() << endl;
        string client = model.getClientName();
        if (!client.empty()) {
            out << "| Клиент: " << client << ", на счете " << model.getClientBalance() << " руб." << endl;
        }
        out << "+-----------------------------------------+" << endl;
    }

private:
    AtmModel& model;
    ostream& out;
};

// Служебная панель оператора: состояние и наличные в банкомате
class OperatorView : public Observer {
public:
    static const int LOW_CASH = 3000;

    OperatorView(AtmModel& model, ostream& out) : model(model), out(out) {
        model.addObserver(this);
    }
```

```
    ~OperatorView() { model.removeObserver(this); }

    void update() override { print(); }

    void print() {
        out << "  [оператор] состояние: " << model.getStateName()
            << " | наличных: " << model.getCash() << " руб."
            << " | операций: " << model.getHistory().size();
        if (model.getCash() < LOW_CASH) out << " | ВНИМАНИЕ: мало денег";
        out << endl;
    }

private:
    AtmModel& model;
    ostream& out;
};

// ======================== Контроллер ========================

// Принимает команды пользователя и вызывает методы модели.
// Сам ничего не выводит о данных - это делают представления.
class AtmController {
public:
    AtmController(AtmModel& model) : model(model) {}

    // Команды: pin <код>, take <сумма>, end, load <сумма>
    // Возвращает false, если команда не распознана
    bool execute(const string& command) {
        istringstream in(command);
        string name;
        in >> name;
        int value = 0;

        if (name == "pin" && in >> value) {
            model.enterPin(value);
        } else if (name == "take" && in >> value) {
            model.withdraw(value);
        } else if (name == "end") {
            model.finish();
        } else if (name == "load" && in >> value) {
            model.loadMoney(value);
        } else {
            return false;
        }
        return true;
    }
```

```
    // Интерактивный режим: читает команды до "exit" или конца ввода
    void run(istream& in, ostream& out) {
        out << "Команды: pin <код>, take <сумма>, end, load <сумма>, exit" << endl;
        string line;
        while (out << "> " && getline(in, line)) {
            if (line == "exit") break;
            if (line.empty()) continue;
            if (!execute(line)) out << "Неизвестная команда" << endl;
        }
        out << endl;
    }

private:
    AtmModel& model;
};

// ======================== Тестирование ========================

class Tester {
public:
    Tester() : passed(0), failed(0) {}

    void check(bool condition, const string& name) {
        if (condition) {
            passed++;
            cout << "  [OK]   " << name << endl;
        } else {
            failed++;
            cout << "  [FAIL] " << name << endl;
        }
    }

    void printResult() const {
        cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
    }

private:
    int passed;
    int failed;
};

// Наблюдатель для тестов: считает оповещения
class CountingObserver : public Observer {
public:
    CountingObserver() : count(0) {}
    void update() override { count++; }
```

```
    int count;
};

bool contains(const string& text, const string& part) {
    return text.find(part) != string::npos;
}

void testModel(Tester& t) {
    AtmModel model("T1", 5000, 0, 1);
    model.addAccount(1234, "Иванов", 10000);
    CountingObserver counter;
    model.addObserver(&counter);

    model.enterPin(1234);
    t.check(counter.count == 1, "Модель оповещает наблюдателя после изменения");
    t.check(model.getClientName() == "Иванов", "Модель хранит текущего клиента");
    model.withdraw(1000);
    t.check(model.getCash() == 4000 && model.getClientBalance() == 9000, "Модель выдает деньги");
    t.check(model.getHistory().size() == 2, "Модель ведет журнал операций");

    model.removeObserver(&counter);
    model.finish();
    t.check(counter.count == 2, "Отписанный наблюдатель не получает оповещений");
}

void testViews(Tester& t) {
    AtmModel model("T1", 2000, 0, 1);
    model.addAccount(1234, "Иванов", 10000);
    ostringstream screenOut, operatorOut;
    ScreenView screen(model, screenOut);
    OperatorView panel(model, operatorOut);

    model.enterPin(1234);
    t.check(contains(screenOut.str(), "PIN-код верный") && contains(screenOut.str(), "Иванов"),
        "Экран показывает сообщение и клиента");
    t.check(contains(operatorOut.str(), "Выполнение операций"),
        "Панель оператора показывает состояние");

    model.withdraw(1500);
    t.check(contains(operatorOut.str(), "ВНИМАНИЕ: мало денег"),
        "Панель оператора предупреждает о нехватке денег");
}

void testController(Tester& t) {
    AtmModel model("T1", 5000, 0, 1);
    model.addAccount(1234, "Иванов", 10000);
    AtmController controller(model);
```

```
    t.check(controller.execute("pin 1234") && model.getStateName() == "Выполнение операций",
        "Команда pin меняет состояние модели");
    t.check(controller.execute("take 2000") && model.getCash() == 3000, "Команда take снимает деньги");
    t.check(controller.execute("end") && model.getStateName() == "Ожидание", "Команда end завершает работу");
    t.check(controller.execute("load 1000") && model.getCash() == 4000, "Команда load загружает деньги");
    t.check(!controller.execute("fly 10") && !controller.execute("take"),
        "Неизвестные и неполные команды отклоняются");

    istringstream input("pin 1234\ntake 500\nexit\ntake 500\n");
    ostringstream output;
    controller.run(input, output);
    t.check(model.getCash() == 3500, "Интерактивный режим останавливается на exit");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testModel(t);
    testViews(t);
    testController(t);
    t.printResult();
}

// ======================== Демонстрация ========================

int main() {
    setlocale(LC_ALL, "Russian");

    random_device device;
    AtmModel model("ATM-001", 5000, 0.1, device());
    model.addAccount(1234, "Иванов", 10000);
    model.addAccount(4321, "Петрова", 3000);

    ScreenView screen(model, cout);
    OperatorView panel(model, cout);
    AtmController controller(model);

    cout << "===== Демонстрация =====" << endl;
    vector<string> script = { "take 1000", "pin 1234", "take 3000", "end", "pin 4321", "take 2000", "load 20000" };
    for (const string& command : script) {
        cout << "\n> " << command << endl;
        controller.execute(command);
    }

    runTests();
```

```
    cout << "\n===== Интерактивный режим =====" << endl;
    cout << "PIN-коды: 1234 (Иванов), 4321 (Петрова)" << endl;
    controller.run(cin, cout);
    return 0;
}
```